

Algorithms and Data Structures I.

Practice 1

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

February 19, 2024

Content in this Practice I

1 Informations

- Contact information
- Lesson information

2 Introduction to Algorithms and Data Types

- Importance of Algorithms
- Algorithms
- Time complexity
- Space complexity
- Data Structures and Data Types
- Data Structure: Array
- Sorting problem

3 Simple Selection algorithm

Content in this Practice II

- Simple Selection algorithm - Introduction
- Simple Selection - Max and Min Selection Sort
- Max Selection Sort - example
- Max Selection Sort - Analysis
- Min Selection Sort - Exercise

4 Simple Exchange algorithm

- Simple Exchange - Introduction
- Simple Exchange - Bubble Sort
- Bubble Sort - example
- Bubble Sort - Analysis
- Improved Bubble Sort - example

5 Simple Insertion algorithm

Content in this Practice III

- Simple Insertion - Introduction
- Simple Insertion - Insertion Sort "Naive"
- Insertion Sort - Example
- Insertion Sort - Analysis
- Insertion Sort - Summary
- Insertion Sort - Exercise

Informations

Contact information

- Name: Gábor Pusztai
- Files: `wornox.web.elte.hu`
- Teams: Gábor Pusztai (AFBDPK)
- Email: `afbdk@inf.elte.hu`
- Useful links:
 - `aszt.inf.elte.hu/~asvanyi/ds/AlgDs1/AlgDs1LectureNotes-2024-02-08.pdf`
 - `aszt.inf.elte.hu/~asvanyi/ds/AD.pdf`

Lesson information

- Personal attendance is mandatory: only 3 skips are allowed
- 2 exams during the class:
 - Exam 1:
 - When: (announced later) around midterm, probably before the 1 week long spring seasonal break
 - Time and location: at this lesson
 - Format: on paper | Score: 60 points (minimum: 20)
 - Exam 2:
 - When: (announced later) around endterm, probably at the last week
 - Time and location: at this lesson
 - Format: on paper | Score: 60 points (minimum: 20)
- The final grade will be determined by the sum of the exams: $60+60=120$. One rewrite opportunity at the end.
(2) 40 - 59, (3) 60 - 79, (4) 80 - 99, (5) 100-120

Introduction to Algorithms and Data Types

Importance of Algorithms

Why is it important to study Algorithms and data structures?

- To solve problems with finite computational resources:
 - time (*"Time is money", "You can get back money after you spend it, but once the time is spent, you can never get it back."*)
 - memory (*nowadays inexpensive, but still not infinite*)
- Optimization:
 - time (*waiting time*)
 - memory (*e.g. cloud space is expensive*)

Algorithms

What is an algorithm?

*An **algorithm** is any well-defined computational procedure that takes some value, or set of values, as input and produces some value, or set of values, as output in a **finite amount of time**.*

*An **algorithm** is thus a sequence of computational steps that transform the input into the output.*

*You can also view an **algorithm** as a tool for solving a well-specified computational problem.*

Time complexity

Time complexity: Reflects some abstract time. We count the subroutine calls + the number of loop iterations during the program's run.

Notations:

- $MT(n)$ (Maximum Time),
- $AT(n)$ (Average or expected Time)
- $mT(n)$ (minimum Time)

$$MT(n) \geq AT(n) \geq mT(n)$$

Typically, the time complexities of the algorithms are not calculated precisely. Only we calculate their *asymptotic order* or make *asymptotic estimation*(s).

Asymptotic estimation(s)

- Asymptotic UPPER estimate:
 - O (big-O, Omicron)
 - runtime is at *most** our given value
- Asymptotic LOWER estimate
 - Ω (Omega)
 - runtime is at *least** our given value
- Asymptotic UPPER AND LOWER estimate
 - Θ (Theta)
 - runtime is "*expected somewhere near*"* our given value

*More precise definitions in Lecture

Space complexity

Space complexity: Is an abstract measure of the algorithm's space (memory) requirements. It depends on the depth of the subroutine calls and the sizes of the data structures.

Notations:

- $MS(n)$ (Maximum Space complexity),
- $AS(n)$ (Average or expected Space complexity)
- $mS(n)$ (minimum Space complexity)

$$MS(n) \geq AS(n) \geq mS(n)$$

Data Structures and Data Types

Data structure:

- store and
- organize data.

Why use different Data Structures?

- different purposes $\rightarrow$ different best solutions

For different Data Structures, we need different operations to work with.

Data Structure: Array

Array:

- The most common datatype
- Direct access to **data** through *indexing*.
- Notations:

$$A : \mathcal{T}[n]$$

- A : Array
- T : Type
- n : The size of A
 - $A.length$
 - The maximum amount of data that can be stored in A .

<i>Index:</i>	0	1	2	3	4
Data:	3	5	2	4	1

Sorting problem

Problem:

We need to sort a sequence of numbers into ascending order!

Example:

- input: (31, 41, 59, 26, 41, 58)
- correct output: (26, 31, 41, 41, 58, 59)

Which algorithm is best for a given application?

It depends on many factors like:

- the number of items to be sorted,
- the extent to which the items are already somewhat sorted (like: 2,1,3,4,5)
- possible restrictions on the item values (like: only positive whole numbers)
- hardware configurations

Sorting problem

Why should we learn "simpler" straight methods instead of the "best" ones?

- Easier to understand at the beginning.
- Easier to implement into code.
- At small n (small amounts of data), they could be faster.

Simple Selection algorithm

Simple Selection - Introduction

- Illustration problem: Sorting my favorite musics in an unsorted playlist.



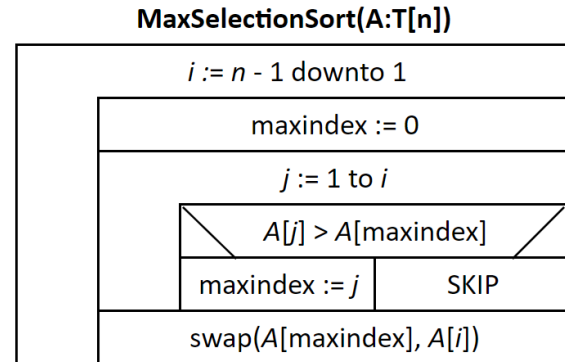
- Idea:
 - Find (*select*) the best song.
 - Put it to the top (first position) of the playlist.
 - Repeat! (second favorite to the second position and so on)

Simple Selection - Max and Min Selection Sort

- 2 types: **Max (or min) Selection Sort**
 - Select the maximum (or minimum) item in the unsorted part.
 - Swap it with the last (or first) positioned item in the unsorted part. (This data will be sorted)
 - Repeat these operations with the remaining unsorted items.
- *Selection sort is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the array and moving it to the sorted portion of the array.*

Max Selection Sort - example

Array:	$\langle 5, 4, 2, 8, 3 \rangle$
1. iteration:	$\langle \underline{5}, 4, 2, \mathbf{8}, \underline{3} \rangle$
2. iteration:	$\langle \mathbf{5}, 4, 2, \underline{3} 8 \rangle$
3. iteration:	$\langle 3, \mathbf{4}, \underline{2} 5, 8 \rangle$
4. iteration:	$\langle \mathbf{3}, \underline{2} 4, 5, 8 \rangle$
5. iteration:	$\langle 2 3, 4, 5, 8 \rangle$
Sorted array:	$\langle 2, 3, 4, 5, 8 \rangle$



Max Selection Sort - Analysis

Time complexity analysis:

- Worst-case: when we need to sort an array in ascending order, but the array is initially in descending order. $O(n^2)$
- Best-case: when the array is already sorted. $\Omega(n^2)$

Space complexity analysis:

- No extra space is required $O(1)$

Min Selection Sort - Exercise

Min selection sort: is the mirror image of the Max selection sort.

Sort the following array with *Min selection sort*!

$\langle 44, 55, 12, 42, 94, 18, 06, 67 \rangle$

Example:

1. iteration: $\langle \underline{44}, 55, 12, 42, 94, 18, \mathbf{06}, 67 \rangle$

2. iteration: $\langle 06 \mid \underline{55}, \mathbf{12}, 42, 94, 18, 44, 67 \rangle$

3. iteration: $\langle 06, 12 \mid \underline{55}, 42, 94, \mathbf{18}, 44, 67 \rangle$

.....

Animation: https://upload.wikimedia.org/wikipedia/commons/3/3e/Sorting_selection_sort_anim.gif

Min Selection Sort - Exercise

Min selection sort: is the mirror image of the Max selection sort.

Sort the following array with *Min selection sort*!

$\langle 44, 55, 12, 42, 94, 18, 06, 67 \rangle$

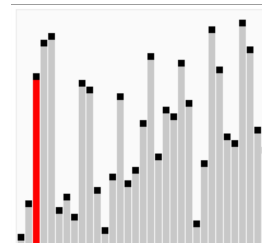
Solution:

Initial keys	<u>44</u>	55	12	42	94	18	06	67
i=1	06	<u>55</u>	12	42	94	18	44	67
i=2	06	12	<u>55</u>	42	94	18	44	67
i=3	06	12	18	<u>42</u>	94	55	44	67
i=4	06	12	18	42	<u>94</u>	55	44	67
i=5	06	12	18	42	44	<u>55</u>	94	67
i=6	06	12	18	42	44	55	<u>94</u>	67
i=7	06	12	18	42	44	55	67	94

Simple Exchange algorithm

Simple Exchange - Introduction

- Illustration problem: I have different items on my desk which I want to sort by height, but I don't know the exact height of each object.



- Idea:
 - Place all the objects next to each other and compare the objects' height to each other
 - Swap if the left is smaller than the right neighbor.
 - Repeat! (Keep swapping until every item is in order)

Simple Exchange - Bubble Sort

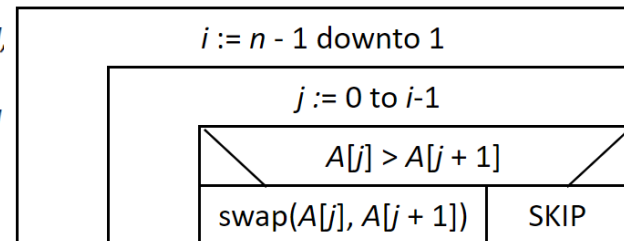
- Bubble Sort algorithm:
 - Traverse from left and compare adjacent 2 elements.
 - Swap if the next element smaller. (The higher one is placed at right side.)
 - In this way, the largest element is moved to the rightmost end at first. (This data will be sorted)
 - Repeat these operations with the remaining unsorted items.
- *Bubble sort is a simple algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order.*

Bubble Sort - example

0	1	2	3	4	swap	
3	5	2	4	1	0	
3	5	2	4	1	1	
3	2	5	4	1	1	
3	2	4	5	1	1	
3	2	4	1	5		1. iteration end, 5 is sorted
3	2	4	1	5	1	
2	3	4	1	5	0	
2	3	4	1	5	1	
2	3	1	4	5		2. iteration end, 4 and 5
2	3	1	4	5	0	
2	3	1	4	5	1	
2	1	3	4	5		3. iteration end,
2	1	3	4	5	1	
1	2	3	4	5		4. iteration end

Overall swap: 7
 Overall comparisons: 10

BubbleSort(A :T[n])



Bubble Sort - Analysis

Time complexity analysis:

- Worst-case: when we need to sort an array in ascending order, but the array is initially in descending order. $O(n^2)$
- Best-case: when the array is already sorted. $\Omega(n^2)$
But we can improve this! $\rightarrow$ *Improved Bubble Sort algorithm*

Space complexity analysis:

- No extra space is required $O(1)$

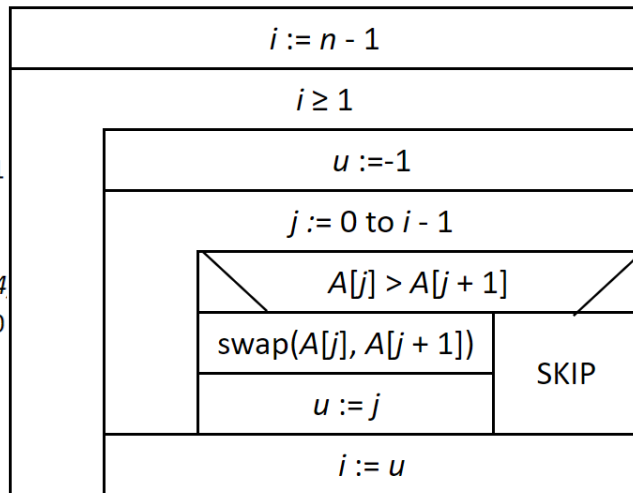
Animation: https://upload.wikimedia.org/wikipedia/commons/5/54/Sorting_bubblesort_anim.gif

Improved Bubble Sort - example

0	1	2	3	4	swap	
2	3	1	4	5	0	
2	3	1	4	5	1	u=1
2	1	3	4	5	0	
2	1	3	4	5	0	
2	1	3	4	5	1	1. iteration end 3,4
2	1	3	4	5	1	u=0
1	2	3	4	5		terminate

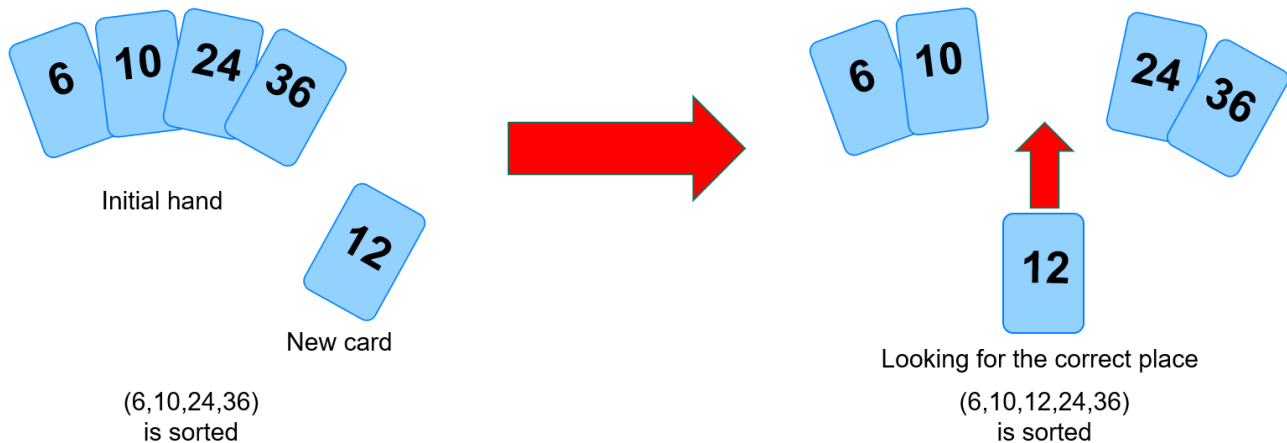
Overall swap: 2
 Overall comparisons: 5

ImprovedBubbleSort(A:T[n])



Simple Insertion algorithm

Simple Insertion - Introduction



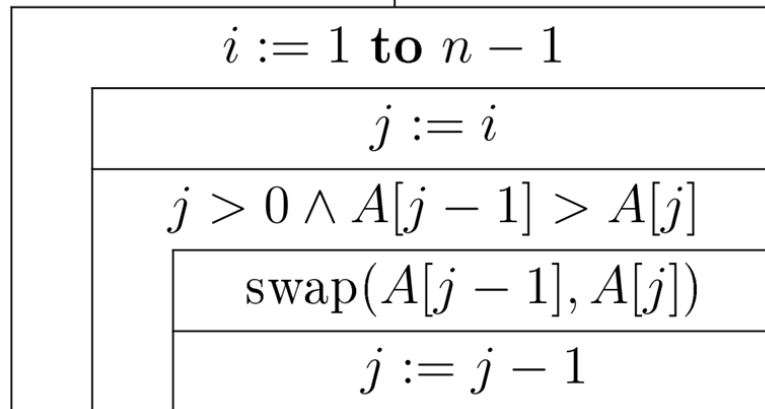
Simple Insertion - Insertion Sort "Naive"

	Array:	$\langle 5, 4, 2, 8, 3 \rangle$	
Before	1. iteration:	$\langle 5, 4, 2, 8, 3 \rangle$	
During	1. iteration:	$\langle 5 4, 2, 8, 3 \rangle$	
After	1. iteration:	$\langle 5 4, 2, 8, 3 \rangle$	
Before	2. iteration:	$\langle 5 4, 2, 8, 3 \rangle$	
During	2. iteration:	$\langle 5, 4 2, 8, 3 \rangle$	<i>Need to sort: swap 5-4</i>
After	2. iteration:	$\langle 4, 5 2, 8, 3 \rangle$	
Before	3. iteration:	$\langle 4, 5 2, 8, 3 \rangle$	
During	3. iteration:	$\langle 4, 5, 2 8, 3 \rangle$	<i>Need to sort: swap 5-2</i>
During	3. iteration:	$\langle 4, 2, 5 8, 3 \rangle$	<i>Swap 4-2</i>
After	3. iteration:	$\langle 2, 4, 5 8, 3 \rangle$	
*After	4. iteration:	$\langle 2, 4, 5, 8 3 \rangle$	<i>*not detailed: 8 is in the correct position</i>
*After	5. iteration:	$\langle 2, 3, 4, 5, 8 \rangle$	<i>*not detailed: multiple swaps required</i>
	Sorted array:	$\langle 2, 3, 4, 5, 8 \rangle$	

Insertion Sort "Naive"

$\langle 5 \mid 4, 2, 8, 3 \rangle$ $i := 1$
 $A[j] = 4$
 Sorted | Unsorted part $A[j-1] = 5$
 $A[j-1] > A[j] \rightarrow \text{swap}()$

naiveInsertionSort($A : \mathcal{T}[n]$)



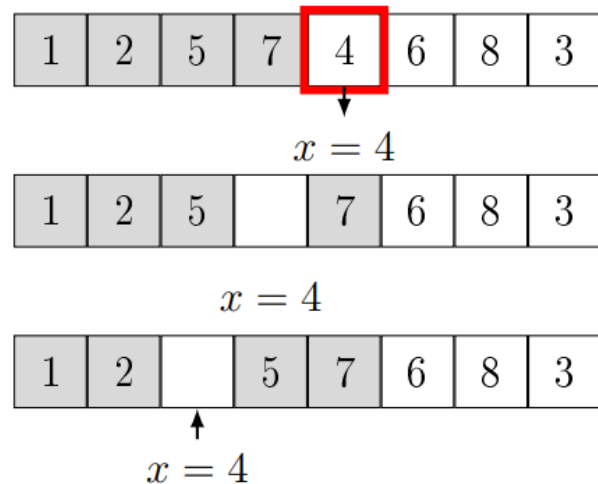
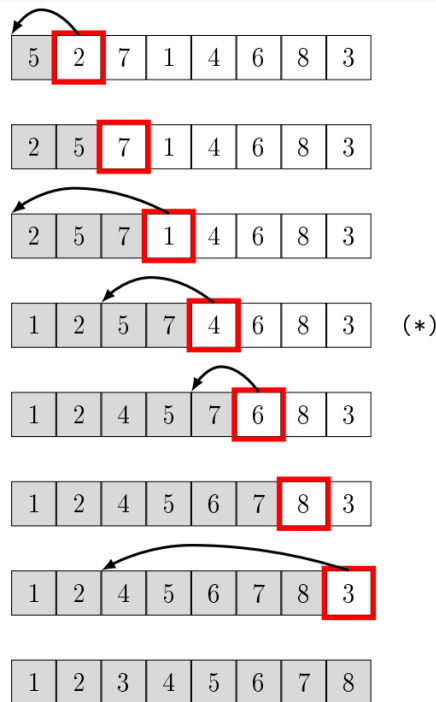
Index: 0 1 2 3 4
 $\langle 5, 4, 2, 8, 3 \rangle$
 $\langle 5 \mid 4, 2, 8, 3 \rangle$
 $\langle 4, 5 \mid 2, 8, 3 \rangle$
 $\langle 2, 4, 5 \mid 8, 3 \rangle$
 $\langle 2, 4, 5, 8 \mid 3 \rangle$
 $\langle 2, 4, 5, 3, 8 \mid \rangle$
 Detailed $\langle 2, 4, 3, 5, 8 \mid \rangle$
 swap() $\langle 2, 3, 4, 5, 8 \mid \rangle$
 steps
 $\langle 2, 3, 4, 5, 8 \rangle$

Insertion Sort "Naive" - Problem

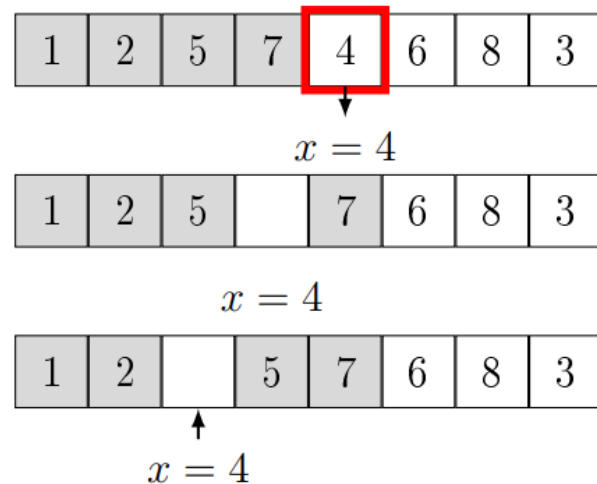
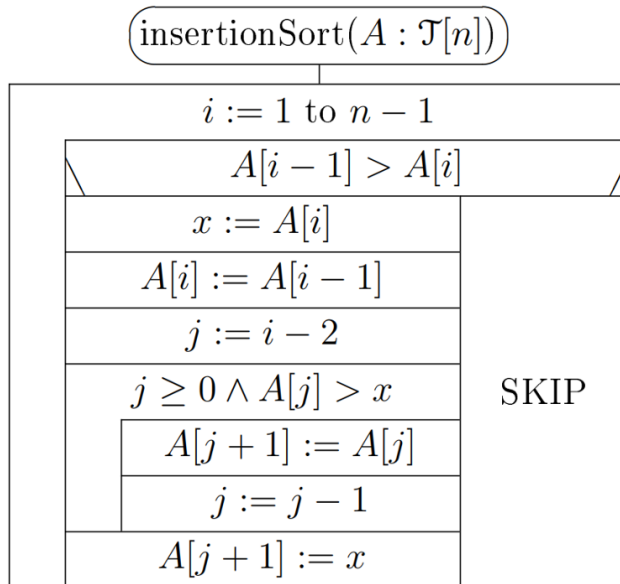
What is the problem with the Insertion sort - Naive?

- Swaps require computational power
- Optimization: Reduce the number of swaps
→ *Insertion sort algorithm*

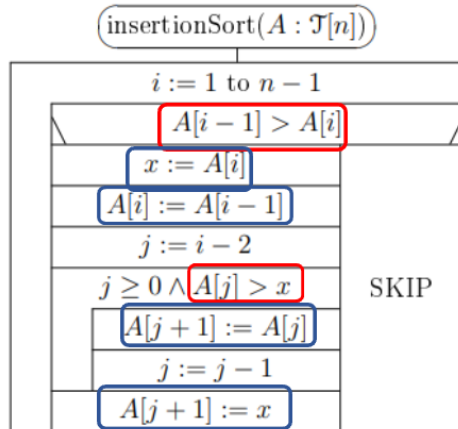
Insertion Sort - Example



Insertion Sort



Insertion Sort - Analysis



		0	1	2	3	4	5	6	7	comp	move
	A	4	3	1	3'	5	2	6	4'		
		0	1	2	3	5	6	7	8		
		>	i								
1. iteration	A	4	3	1	3'	5	2	6	4'	1	3
		>	>	i							
2. iteration	A	3	4	1	3'	5	2	6	4'	2	4
		$\mathcal{E}$	>	i							
3. iteration	A	1	3	4	3'	5	2	6	4'	2	3
		$\mathcal{E}$	$\mathcal{E}$	i							
4. iteration	A	1	3	3'	4	5	2	6	4'	1	0
		$\mathcal{E}$	>	>	>	>	i				
5. iteration	A	1	3	3'	4	5	2	6	4'	5	6
		$\mathcal{E}$	$\mathcal{E}$	i							
6. iteration	A	1	2	3	3'	4	5	6	4'	1	0
		$\mathcal{E}$	>	>							
7. iteration	A	1	2	3	3'	4	5	6	4'	3	4
Overall:	A	1	2	3	3'	4	4'	5	6	15	20

Insertion Sort - Summary

Time complexity:

- $mT(n) \in \Theta(n)$
- $AT(n), MT(n) \in \Theta(n^2)$

Space complexity

- $\Theta(1)$
- Only a few temporal variables.

Stable: The sorting algorithms maintain the relative order of the duplicate data.

Insertion Sort - Exercise

Sort the following array with *Insertion sort*!

$\langle 6, 5, 3, 1, 8, 7, 2, 4 \rangle$

Example:

1. iteration: $\langle 6 \mid \underline{5}, 3, 1, 8, 7, 2, 4 \rangle$

2. iteration: $\langle 5, 6 \mid \underline{3}, 1, 8, 7, 2, 4 \rangle$

3. iteration: $\langle 3, 5, 6 \mid \underline{1}, 8, 7, 2, 4 \rangle$

.....

Insertion Sort - Exercise

Sort the following array with *Insertion sort*!

$\langle 6, 5, 3, 1, 8, 7, 2, 4 \rangle$

Solution (animation):

[https://en.wikipedia.org/wiki/Insertion_sort#/media/](https://en.wikipedia.org/wiki/Insertion_sort#/media/File:Insertion-sort-example-300px.gif)

[File:Insertion-sort-example-300px.gif](https://en.wikipedia.org/wiki/Insertion_sort#/media/File:Insertion-sort-example-300px.gif)

Informations
Introduction to Algorithms and Data Types
Simple Selection algorithm
Simple Exchange algorithm
Simple Insertion algorithm

Simple Insertion - Introduction
Simple Insertion - Insertion Sort "Naive"
Insertion Sort - Example
Insertion Sort - Analysis
Insertion Sort - Summary
Insertion Sort - Exercise

Ending

Thank you for your attention!